Supplementary Materials

[Authors]

February 2026

Comparing ML-Specific and General Python Code Smells Across Project Characteristics

ICSME 2026

Abstract
This document provides supplementary materials for the paper "Comparing ML-Specific and General Python Code Smells Across Project Characteristics." It includes detailed information about dataset construction, domain and CI/CD classification methodologies, code smell detection procedures, statistical analysis methods, and comprehensive results for all research questions.

---

1 Overview

Our study presents a large-scale empirical study investigating the relationship between ML-specific code smells and general Python code smells across 279 open-source ML projects, aims to uncover how these two types of smells correlate differently with key project attributes, such as size and complexity. By systematically comparing their prevalence within the same set of repositories, we provide evidence-based guidance for ML practitioners on whether to prioritize quality assurance efforts on ML-specific defects or traditional software engineering concerns based on their unique project context.

2 Dataset Construction

2.1 NICHE Update Process

The basis of our research is the NICHE dataset, a manually curated collection of 572 Python-based machine learning projects. Since the original metadata reflected a snapshot from 2020, we updated the dataset in September 2025. We implemented a hybrid data collection pipeline combining static source code analysis with dynamic metadata retrieval via the GitHub GraphQL API. We utilized the `cloc` to quantify Lines of Code (LOC) exclusively for Python source files. Ultimately, 565 out of 572 repositories (98.8%) were successfully reached, with seven projects excluded due to being deleted or inaccessible.

2.2 Filtering Criteria

To exclude abandoned or experimental repositories and focus on engineered software processes, we applied a multi-dimensional filtering pipeline:

- **Recent Commit Activity:** Restricted to repositories with activity in 2024 or later.
- **Development History:** Minimum of 100 commits to ensure project maturity.
- **Community Validation:** Popularity threshold as 100 GitHub stars.
- **Collaborative Engineering:** Minimum of 5 contributors to focus on collaborative repositories.
- **Workflow Complexity:** Minimum of 2 branches as a sign of structured release management.

After applying these criteria and excluding two repositories without Python code our final dataset consists of **279 repositories**.

2.3 Feature Enrichment

The dataset was enriched with updated metadata, including the presence of CI/CD pipelines, application domains, last commit dates, branch counts, and contributor counts. Project age was explicitly calculated as the time elapsed from each repository's creation date to the analysis cutoff date on December 4, 2025. To ensure representative sampling across scales, projects were stratified by Lines of Code (LOC) into Small ($< 10,445$), Medium ($10,445 - 26,821$), and Large ($> 26,821$) categories based on percentile distributions.

3 Domain Classification

3.1 Seven Domains Defined

We classified the projects into seven distinct machine learning application domains:

- ML Frameworks/Libraries
- Natural Language Processing (NLP)
- Computer Vision (CV)
- Domain-Specific Applications

- Data Science/Traditional ML
- MLOps/Deployment
- Reinforcement Learning (RL)

Domain-Specific Applications includes specialized projects such as bio-informatics, audio signal processing, and smart home IoT systems.

3.2 Keywords per Domain

Representative keywords were assigned to each domain to facilitate identification. These keywords are based on a literature review and were utilized in the keyword-matching process for automated domain classification:

- **Computer Vision:** yolo, mask-rcnn, opencv, segmentation.
- **NLP:** transformer, bert, gpt, sentiment, ner.
- **Reinforcement Learning:** dqn, ppo, a3c, openai-gym, q-learning.
- **MLOps/Deployment:** MLflow, kubeflow, docker, kubernetes.
- **ML Frameworks:** tensorflow, pytorch, keras, jax.
- **Data Science/Traditional ML:** scikit-learn, xgboost, autoML, pandas.
- **Domain-Specific Application:** audio, medical-imaging, shap, time-series.

3.3 Automated Classification

The first phase used automated keyword-based matching via the GitHub REST API to analyze repository descriptions, README content, and project tags. The logic successfully assigned initial categories to 81.1% of the repositories (60.7% via priority indicators and 20.4% via weighted scoring).

3.4 Manual Review Process

The second stage involved a manual review of the remaining 53 repositories (18.9%) that scored below automated thresholds or showed multi-domain characteristics. This manual validation ensured 100% classification coverage across the final dataset.

3.5 Validation Results

The final distribution shows ML Frameworks as the largest group (34.1%), followed by NLP (14.0%) and CV (13.3%). MLOps projects were found to be concentrated in the "Large" size category, indicating higher architectural complexity. Table 1 details the full distribution of projects across all domains.

Table 1: Distribution of Projects by ML Domain

ML Domain	N	Percentage
ML Frameworks / Libraries	95	34.1%
Natural Language Processing	39	14.0%
Computer Vision	37	13.3%
Domain-Specific Applications	33	11.8%
Data Science / Traditional ML	27	9.7%
MLOps / Deployment	26	9.3%
Reinforcement Learning	22	7.9%
Total	**279**	**100.0%**

4 CI/CD Classification

4.1 4.1 Detection Methodology

We moved beyond the textual descriptions in the original NICHE dataset to a binary classification (Yes/No). We defined CI/CD adoption as the automation of build, test, or deployment processes. A custom Python script verified the existence of configuration files (GitHub Actions, Travis CI, Jenkins, etc.) and ensured they contained non-empty executable commands.

4.2 Validation Process

Manual validation of a random sample (30 projects) confirmed the reliability of our automated script. The validation achieved 100% accuracy for projects with CI/CD and 93.3% for those without. One project was reclassified after manual inspection revealed a custom Bazel build system not initially detected by the script.

4.3 Final Distribution

After validation, the final results showed that **244 projects (87.5%)** utilized CI/CD pipelines, while **35 projects (12.5%)** did not. Security-only and documentation-only workflows were correctly excluded.

5 Code Smell Detection

5.1 CodeSmile Configuration

To analyze ML-specific code quality, we utilized the CodeSmile tool. For the purposes of this study, we focused exclusively on the 12 ML-specific code smells that have been formally validated in literature.

Table 2: The 12 Validated ML-Specific Code Smells

Quality Attribute	ML-Specific Code Smells
Reliability	Columns and DataType Not Explicitly Set, DataFrame Conversion API Misused, In-Place APIs Misused, Gradients Not Cleared Before Backward Propagation, NaN Equivalence Comparison Misused, PyTorch Call Method Misused.
Performance	Chain Indexing, Unnecessary Iteration, TensorArray Not Used, Memory Not Freed.
Maintainability	Matrix Multiplication API Misused, Merge API Parameter Not Explicitly Set.

The following smells available in the CodeSmile repository were intentionally were excluded as they lack validation in main empirical studies and do not currently have widely accepted definitions in established literature.

- Broadcasting Feature Not Used
- Deterministic Algorithm Option Not Used
- Empty Column Misinitialization
- Hyperparameters Not Explicitly Set

5.2 Pylint Configuration

We analyzed the top 20 general Python smells identified in the dataset. To focus exclusively on structural and logic-based smells, we performed extensive noise filtering.

Table 3: Top 20 General Python Smells Detected

1	unused-argument	6	redefined-outer-name	11	attribute-defined-outside-init	16	too-many-instance-attribu
2	no-member	7	too-many-locals	12	abstract-method	17	redefined-builtin
3	protected-access	8	unsubscriptable-object	13	no-else-return	18	arguments-differ
4	no-self-use	9	syntax-error	14	unused-variable	19	logging-fstring-interpolati
5	too-many-arguments	10	too-few-public-methods	15	fixme	20	no-value-for-parameter

Noise Filtering (Excluded Pylint Errors):

- **Style/Naming:** invalid-name, line-too-long, trailing-whitespace, missing-final-newline, trailing-newlines, bad-indentation, super-with-arguments, useless-object-inheritance, bad-continuation, bad-whitespace, bad-option-value.
- **Docstrings:** missing-module-docstring, missing-class-docstring, missing-function-docstring, empty-docstring.
- **Imports/Environment:** import-error, no-name-in-module, unable-to-import, unused-import, wildcard-import, reimported, cyclic-import, relative-beyond-top-level, import-outside-toplevel, ungrouped-imports, useless-import-alias, multiple-imports, wrong-import-position, wrong-import-order, unused-wildcard-import.

5.3 5.3 Execution Details

The CodeSmile analysis was executed on **November 22, 2025**, and the Pylint analysis was completed on **December 5, 2025**. During execution, large-scale repositories such as *home-assistant/core* and *dmlc/dgl* required excessive runtime (approximately 5–6 hours each) due to high file counts. All projects were successfully analyzed using Pylint; however, CodeSmile identified no ML-specific code smells in **31 repositories**.

6 Statistical Analysis

6.1 RQ1: Comparing ML-Specific and General Python Code Smells

This RQ compares the density of ML-specific smells against general Python smells.

- **Hypothesis ($H_{0.0}$):** The null hypothesis states there is no significant difference between ML-specific and general Python code smell densities within the same project.
- **Statistical Test:** We employed the **Wilcoxon signed-rank test**, a non-parametric paired test suitable for related samples.
- **Effect Size:** Cliff's Delta (δ) was used to quantify the magnitude of the difference (Negligible: < 0.15, Small: $0.15 \leq |\delta| < 0.33$, Medium: $0.33 \leq |\delta| < 0.47$, Large: ≥ 0.47).

6.2 RQ2: ML-Specific Smells and Project Characteristics

We investigated how 12 ML-specific smells correlate with project features using hypotheses $H_{1.0}$ through $H_{6.1}$.

- **Correlation Tests ($H_{1.0}, H_{2.0}, H_{3.0}, H_{4.0}$): Spearman's rank correlation (ρ)** was used to test relationships with LOC, Age, Contributors, and Commit Frequency.
- **Group Comparisons ($H_{1.1}, H_{2.1}, H_{3.1}, H_{4.1}, H_{5.0}, H_{6.0}$):** For binary groups (e.g., CI/CD vs. Non-CI/CD), we used the **Mann-Whitney U test**. For multi-group comparisons (Size Categories and Domains), we used the **Kruskal-Wallis H test** followed by post-hoc pairwise **Dunn's tests**.
- **Association Test ($H_{6.1}$):** A **Chi-square (χ^2)** test of independence was used to find associations between specific domains and smell types, measured by **Cramér's V**.

6.3 RQ3: General Python Smells and Project Characteristics

To maintain consistency, we applied the same statistical methodology as RQ2 to the top 20 general Python code smells.

- **Correlation Tests ($H_{1.0}, H_{2.0}, H_{3.0}, H_{4.0}$):** We utilized **Spearman's rank correlation (ρ)** to test the null hypotheses that there is no significant relationship between general Python smell density and project Size (LOC), Age, Contributors, or Commit Frequency.
- **Group Comparisons ($H_{1.1}, H_{2.1}, H_{3.1}, H_{4.1}, H_{5.0}$):** We employed the **Mann-Whitney U test** for binary comparisons (Age, Team Size, Activity, CI/CD) and the **Kruskal-Wallis H test** for the multi-group Size category comparison ($H_{1.1}$) to test if smell density differs significantly between groups. Effect sizes were measured using **Cliff's Delta (δ)**.

- **Domain Analysis ($H_{6.0}$):** We used the **Kruskal-Wallis H test** to determine if general Python smell density varies significantly across the seven defined ML domains.

6.4 RQ4: Differential Analysis of Correlations

This RQ analyzes whether project characteristics impact ML and Python smells differently using hypotheses $H_{7.0}$ through $H_{12.0}$.

- **Comparing Correlations ($H_{7.0}$–$H_{10.0}$):** We used **Fisher's z-transformation** to compare the Spearman correlation coefficients calculated in RQ2 and RQ3 for Size, Age, Contributors, and Commit Frequency.
- **CI/CD Effect Size Comparison ($H_{11.0}$):** We compared the Cliff's Delta values from the CI/CD analysis using **Bootstrap confidence intervals** (1000 iterations) to determine if automation impacts one smell type more significantly than the other.
- **Domain Ranking Association ($H_{12.0}$): Spearman's** ρ was applied to the median density rankings of domains for both smell types to identify concordant or discordant quality patterns across fields.

6.5 Statistical Parameters and Thresholds

For all categorical and grouping analyses, the following specific median thresholds and significance corrections were applied:

- **Median Split for Age:** 7.56 years.
- **Median Split for Team Size:** 53 contributors.
- **Median Split for Activity:** 15.78 commits per month.
- **Bonferroni Correction (α):** For post-hoc domain comparisons (7 domains), a significance level of **0.0024** was utilized. For size category comparisons (3 groups), a corrected α of **0.017** was applied.

7 Results Summary

7.1 RQ1: Comparative Analysis

The objective of RQ1 was to compare the prevalence and density of ML-specific code smells against general Python code smells. Statistical testing confirms a massive disparity: traditional technical debt (Python smells) is significantly more prevalent than ML-specific issues across all project scales.

Table 4: Median Smell Densities by Project Size

Project Size	ML-Specific Density	General Python Density
Small (n=82)	0.919	37.379
Medium (n=85)	0.642	42.821
Large (n=112)	0.389	36.481
Overall Median	**0.558**	**38.417**

Table 5: Wilcoxon Signed-Rank Test Results

Wilcoxon W	p-value	Cliff's Delta (δ)	Result
0.0	5.137×10^{-47}	-0.9788 (Large)	Reject $H_{0.0}$

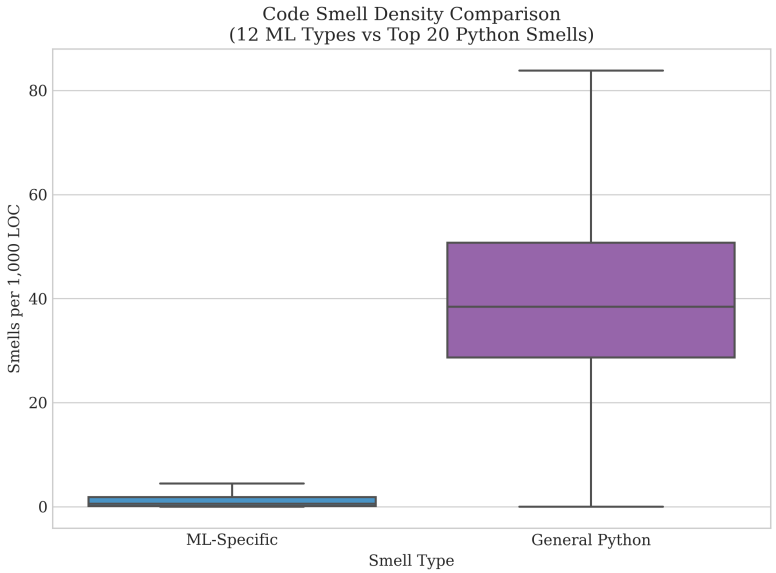

Figure 1: Side-by-side box plots of ML-specific versus general Python smell densities.

7.1.1 Key Findings for RQ1

- General Python smell density is nearly 70 times higher than ML-specific density (38.417 vs 0.558).
- With a p-value of 5.13×10^{-47} and a Cliff's Delta of -0.9788, the difference in density distributions is statistically significant with a nearly perfect effect size.
- While general Python smells stay consistently high, ML-specific smell density decreases as projects grow larger, dropping from 0.919 in small projects to 0.389 in large ones.

7.2 RQ2: ML-Specific Code Smells

The analysis for RQ2 investigated how ML-specific code smells relate to project size, age, collaboration, activity, and domain. While normalized density remains largely independent of project scale and age, significant results were observed in **Commit Frequency** and **Domain Classification**, indicating that development pace and application field are the strongest predictors of ML-specific code smell prevalence.

Table 6: Summary of Correlation Results (ML-Specific Smells)

Hypothesis	**Variable**	**Spearman's ρ**	**p-value**	**Result**
$H_{1.0}$	Size (LOC) vs Density	-0.0707	0.2395	Fail to Reject
$H_{2.0}$	Project Age vs Density	-0.0348	0.5620	Fail to Reject
$H_{3.0}$	Contributors vs Density	-0.1073	0.0736	Fail to Reject
$H_{4.0}$	Commit Freq. vs Density	-0.1549	0.0095	**Reject** H_0

The group comparison analysis further supports the resilience of ML smell density against most project factors, with the exception of activity levels, where high-frequency commit activity correlates with lower smell density.

Domain analysis revealed that Data Science and Traditional ML projects exhibit significantly higher smell densities compared to other fields, likely due to the experimental nature of scripts in those domains.

7.3 RQ3: General Python Smells

We found **no statistically significant correlations or differences** across any of the project dimensions tested (Size, Age, Contributors, Activity, CI/CD, or Domain). This suggests that general technical debt is a systemic issue in ML engineering that is not mitigated by project maturity, team size, or automation.

Table 7: Summary of Group Comparison Results (ML-Specific Smells)

Hyp.	Grouping Factor	Median Density	U / H Stat	p-value	Cliff's δ
$H_{1.1}$	Size (S/M/L)	0.91 / 0.64 / 0.38	3.574 (H)	0.1674	Negligible
$H_{2.1}$	Age (Young/Mature)	0.600 / 0.537	10541.5	0.2280	0.0834 (N)
$H_{3.1}$	Team (Small/Large)	0.806 / 0.412	10690.5	0.1530	0.0987 (N)
$H_{4.1}$	Activity (Low/High)	0.778 / 0.403	11169.5	0.0323	0.1479 (N)
$H_{5.0}$	CI/CD (Yes/No)	0.532 / 0.737	3745.0	0.2390	-0.1230 (N)

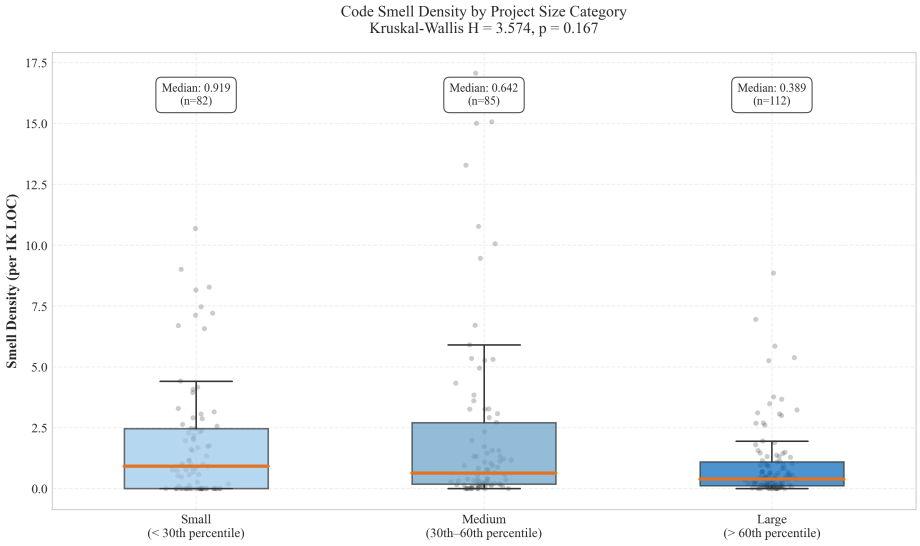

Figure 2: Box plots of ML-Specific Smell Density by Size Category ($H_{1.1}$).

Table 8: Domain Medians and Rankings ($H_{6.0}$ - ML Specific)

Rank	Domain	n	Median Density
1	Data Science / Traditional ML	27	1.955
2	Domain-Specific Applications	33	0.831
3	Computer Vision	37	0.661
4	MLOps / Deployment	26	0.492
5	Natural Language Processing	39	0.487
6	ML Frameworks / Libraries	95	0.379
7	Reinforcement Learning	22	0.370

Table 9: Significant Domain Pairwise Comparisons ($H_{6.0}$)

Comparison Pair	p-value (adj)	Cliff's δ
DS/Trad ML vs ML Frameworks	2.71×10^{-6}	0.593 (Large)
DS/Trad ML vs NLP	2.16×10^{-3}	0.447 (Medium)

Table 10: Chi-Square Association Results ($H_{6.1}$)

Chi-Square (χ^2)	p-value	Cramér's V	Result
3802.34	0.0000	0.200	**Reject** H_0

The group comparison results reinforce the findings from the correlation analysis, showing negligible effect sizes across all categories.

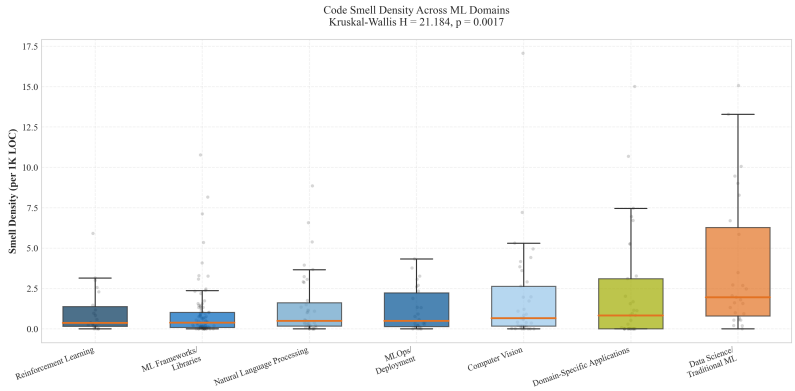

Figure 3: Box plots of ML-Specific Smell Density by Domain ($H_{6.0}$).

Table 11: Summary of Correlation Results (General Python Smells)

Hypothesis	**Variable**	**Spearman's ρ**	**p-value**	**Result**
$H_{1.0}$	Size (LOC) vs Density	0.0083	0.8900	Fail to Reject
$H_{2.0}$	Project Age vs Density	0.0560	0.3520	Fail to Reject
$H_{3.0}$	Contributors vs Density	-0.0255	0.6720	Fail to Reject
$H_{4.0}$	Commit Freq. vs Density	-0.0244	0.6840	Fail to Reject

Table 12: Summary of Group Comparison Results (General Python Smells)

Hyp.	**Grouping Factor**	**Median Density**	**U / H Stat**	**p-value**	**Cliff's δ**
$H_{1.1}$	Size (S/M/L)	37.38 / 42.82 / 36.48	4.328 (H)	0.1150	Negligible
$H_{2.1}$	Age (Young/Mature)	36.805 / 40.447	9178.0	0.4130	-0.0567 (N)
$H_{3.1}$	Team (Small/Large)	40.391 / 36.828	10290.0	0.4060	0.0576 (N)
$H_{4.1}$	Activity (Low/High)	37.959 / 38.765	9940.0	0.7560	0.0216 (N)
$H_{5.0}$	CI/CD (Yes/No)	37.454 / 42.325	3719.0	0.2170	-0.1290 (N)

Unlike ML-specific smells, general Python smells did not vary significantly by domain. Computer Vision had the highest median density, but the differences were not statistically significant ($p = 0.504$).

Table 13: Domain Medians and Rankings (General Python Smells)

Rank	**Domain**	**Median Density**	**Count (n)**
1	Computer Vision	42.325	37
2	Domain-Specific Applications	41.881	33
3	MLOps / Deployment	41.794	26
4	Data Science / Traditional ML	40.901	27
5	Reinforcement Learning	38.748	22
6	Natural Language Processing	37.647	39
7	ML Frameworks / Libraries	35.380	95

7.4 RQ4: Differential Analysis

The final research question examined whether project characteristics influence ML-specific and general Python code smells differently. Using Fisher's z-transformation, we compared the correlation coefficients derived in RQ2 and RQ3.

We found **no statistically significant difference** in how project size, age, or team size correlates with either smell type. In all cases, the difference in correlation strength ($\Delta\rho$) was negligible ($p > 0.05$). This indicates that

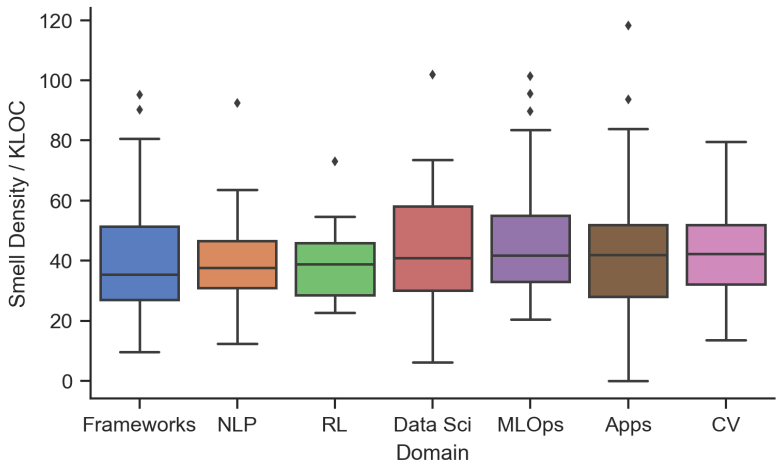

Figure 4: Box plots of General Python Smell Density by Domain ($H_{6.0}$). Note the uniform distribution across domains compared to ML-specific smells.

the "immune" nature of ML smell density and the "persistent" nature of Python debt are consistent across these dimensions.

CI/CD Effect: A bootstrap analysis of Cliff's Delta values revealed that the impact of CI/CD is virtually identical for both smell types ($\Delta\delta = 0.0075$), suggesting that current automation strategies are equally (in)effective for both categories.

Domain Patterns: The most distinct finding comes from the domain ranking analysis ($H_{12.0}$). While not statistically significant overall ($p = 0.119$), specific domains showed "discordant" patterns. For instance, **Data Science** projects ranked highest for ML-specific smell density (Rank 1) but appeared much healthier regarding general Python smells (Rank 4), highlighting a unique trade-off where experimental code quality suffers specifically in ML logic rather than general structure.

Table 14: Fisher's z-Transformation Results (Correlations Comparison)

Hyp.	**Comparison**	ρ_{ML}	ρ_{Py}	**z-score**	**p-value**	**Decision**
$H_{7.0}$	Size (LOC)	-0.0707	0.0083	-0.9287	0.3530	Fail to Reject
$H_{8.0}$	Age	-0.0348	0.0560	-1.0676	0.2857	Fail to Reject
$H_{9.0}$	Contributors	-0.1073	-0.0255	-0.9659	0.3341	Fail to Reject
$H_{10.0}$	Commit Freq.	-0.1549	-0.0244	-1.5477	0.1217	Fail to Reject

Table 15: CI/CD Effect Size Comparison (Bootstrap Test)

δ_{ML}	δ_{Python}	**Diff ($\lvert\Delta\delta\rvert$)**	**95% CI**	**p-value**	**Result**
-0.1230	-0.1304	0.0075	[-0.257, 0.284]	¿ 0.05	Fail to Reject

Table 16: Domain Rankings Comparison ($H_{12.0}$)

Rank	Domain	ML Median	Py Median	$Rank_{ML}$	$Rank_{Py}$
1	Data Science / Traditional ML	1.955	40.901	1	4
2	Domain-Specific Applications	0.831	41.881	2	2
3	Computer Vision	0.661	42.325	3	1
4	MLOps / Deployment	0.492	41.794	4	3
5	Natural Language Processing	0.487	37.647	5	6
6	ML Frameworks / Libraries	0.379	35.380	6	7
7	Reinforcement Learning	0.370	38.748	7	5

Table 17: Discordant Domains Analysis (Top 3 Pairs)

Domain	Rank Diff	Pattern	Implication
Data Science	-3	High ML / Low Py	Focus on ML-Specific Tools
Computer Vision	+2	Low ML / High Py	Focus on General Linters
Reinforcement Learning	+2	Low ML / High Py	Focus on General Linters

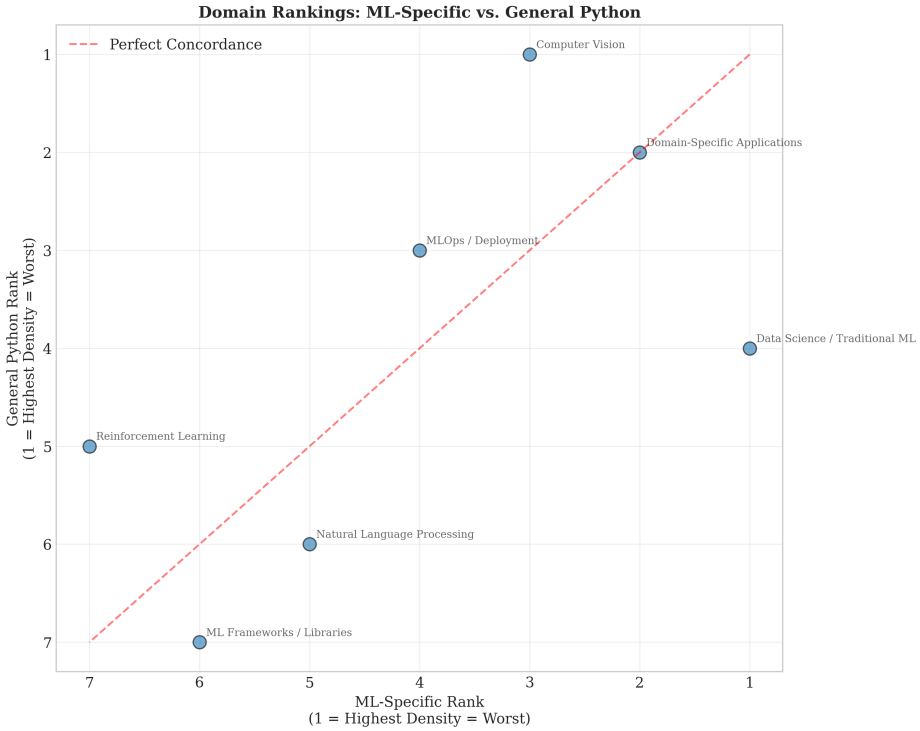

Figure 5: Scatter plot of Domain Rankings: ML-Specific Rank (x-axis) vs. Python Rank (y-axis). Points far from the diagonal (like Data Science) indicate discordant quality patterns.